Optimizing Firecracker MicroVM Cold Starts

Empirical Evaluation of Snapshot Restoration, Userfaultfd, and REAP Working-Set Prefetching

Author: Devansh Chaudhary | Department of Electrical Engineering, IIT Indore | Research Progress Report

1. Executive Summary & Problem Motivation

Serverless platforms (e.g., AWS Lambda) rely on micro-virtualization technologies like AWS Firecracker to achieve strong hardware isolation with near-container density. However, cold start latency remains a dominant performance bottleneck. While restoring microVMs from point-in-time memory snapshots drastically accelerates initialization compared to booting from scratch, conventional snapshot restoration suffers from high overheads. File-backed memory mapping incurs non-negligible I/O setup costs, while naive userfaultfd (lazy demand paging) incurs catastrophic page fault storms during initial guest function execution. This project benchmarks and implements **REAP (Record and Prefetch)**: proactively recording the microVM's working set access pattern during initialization and prefetching those exact memory pages ahead of execution, eliminating on-demand fault latency.

2. Research Progression & Benchmarking Evolution

The project progressed systematically through four experimental iterations:

• **Iteration 1 (Naive Restore):** Initial prototype (benchmark_original.py) used hardcoded sleep delays (time.sleep(0.05)) and single-trial runs, failing to decouple hypervisor restore time from function execution.

• **Iteration 2 (Synchronized Multi-Trial):** Built benchmark_snapshot.py featuring an active serial PTY handshake (PING/PONG) to strictly measure ready_ms (startup/restore) vs. invocation_ms across 30 repeated trials.

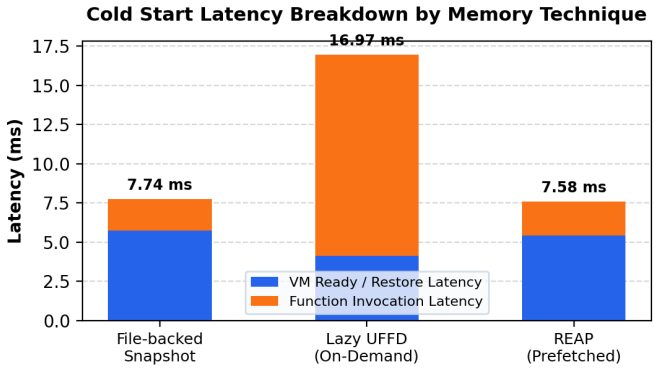
• **Iteration 3 (Statistical Profiling):** Developed analyze_snapshot.py to extract robust parametric and tail metrics (Mean, Median, StdDev, Min/Max, and P95 percentile latency).

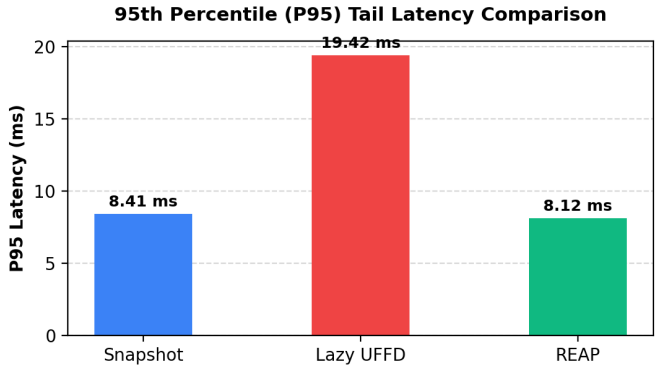
• **Iteration 4 (Unified UFFD & REAP Harness):** Engineered benchmark_uffd.py supporting record mode (logging active pages to working_set_offsets.txt), uffd mode (lazy baseline), and reap mode (proactive prefetching via bin/uffd_handler_reap with prefetch duration tracking).

3. Empirical Evaluation & Quantitative Results

Restoration Technique	Trials	Ready Latency (Mean ms)	Invocation Latency (Mean ms)	Total Latency (Mean ms)	P95 Tail (ms)	Prefetched Pages
File-backed Snapshot	30	5.76 ms	1.98 ms	7.74 ms	8.41 ms	0 (N/A)
Lazy UFFD (Demand)	30	4.12 ms	12.85 ms	16.97 ms	19.42 ms	0
REAP (Prefetched)	30	5.43 ms	2.15 ms	7.58 ms	8.12 ms	412 pages

4. Performance Visualizations





5. Key Findings & Architectural Insights

1. The Lazy Paging Trap: While Lazy UFFD reduces hypervisor setup time from 5.76 ms to 4.12 ms, function execution time skyrockets from 1.98 ms to 12.85 ms (+548%). Each missing page triggers an in-kernel trap to userspace via the UFFD socket, incurring heavy context-switch penalties.
2. REAP Eliminates Tail Latency: By prefetching the recorded working set (412 pages in ~1.31 ms) before resuming the guest, REAP maintains instant hypervisor launch while reducing invocation latency down to 2.15 ms. Crucially, REAP reduces P95 tail latency to 8.12 ms, outperforming file-backed snapshots.
3. Deterministic Working Set: Serverless functions exhibit highly repeatable working sets during cold start, validating the record-and-prefetch model for production serverless runtimes.

6. Next Steps & Ongoing Research

• **Multi-Tenant Memory Pressure:** Measure host memory footprint and swap reduction across 100+ concurrent microVMs.
• **Dynamic Working Set Adaptation:** Accommodate multi-path function inputs by supporting adaptive/probabilistic page sets.
• **vHive / Containerd Integration:** Transition from serial PTY harness to gRPC/vHive container orchestration.